

Compiler Design — Lecture 1 (part 1)

Introduction to Compilers

Purpose: Compiler Design Fundamentals + GATE 2027

1. Programming Language Implementation

A programming language needs some mechanism to convert or process a program so that it can be executed by a computer.

There are two major approaches:

1. **Compiler**
 2. **Interpreter**
-

2. Compiler

A **compiler** takes a source program and translates it into another program, called the **target program**.

Source Program



Compiler



Target Program



Execution

The target program may be:

- Machine code
- Assembly code
- Bytecode
- Another programming language

The important idea is that **translation is performed before the target program is executed**.

Therefore, compilation can be viewed as an **offline** process.

Advantage

Once the target/executable program has been generated, it can generally be executed with different inputs without recompiling the source program.

3. Interpreter

An **interpreter** processes a program as part of its execution.

Source Program + Input



Interpreter



Output

Unlike the compiler approach, there is no requirement that the interpreter first produce a separate executable target program.

Therefore, interpretation can be viewed as an **online** process.

4. Compiler vs Interpreter

Compiler	Interpreter
Translates source program into a target program	Processes source program as part of execution
Translation occurs before execution	Processing occurs during execution
Generally considered offline	Generally considered online
Generated target program can be reused for different inputs	Interpretation remains part of execution

GATE Point

Do **not** memorize that a compiler always produces machine code.

A compiler can generate **assembly, machine code, bytecode, or another language**.

5. Historical Motivation

During the early development of computers, programming was difficult and expensive because programmers had to work close to the hardware.

The development of higher-level programming systems aimed to improve:

- Programmer productivity
 - Ease of programming
 - Abstraction from machine details
-

6. Speedcoding

John Backus developed **Speedcoding**, an early interpreted programming system.

Its main benefit was improved **programmer productivity**.

However, it was significantly slower than handwritten machine-level programs.

This highlighted an important challenge:

How can we make programming easier without sacrificing too much execution efficiency?

This motivated the development of efficient compilers for high-level languages.

7. FORTRAN

John Backus later led the development of **FORTRAN**.

FORTRAN = FORMula TRANslation

Development period:

1954–1957

FORTRAN became one of the first highly successful **high-level programming languages**.

It was particularly important for **scientific and numerical computing**.

Importance

FORTRAN demonstrated that:

High-level programming could improve programmer productivity while a compiler could generate efficient programs.

It was an important milestone in the development of compiler technology and programming languages.

8. Five Major Phases of a Compiler

The Stanford lecture introduces five major phases:

Source Program



Lexical Analysis



Syntax Analysis



Semantic Analysis



Optimization



Code Generation



Target Program

Overview

Phase	Basic Purpose
Lexical Analysis	Converts source text into tokens
Syntax Analysis	Determines syntactic structure
Semantic Analysis	Checks semantic consistency
Optimization	Improves efficiency while preserving meaning
Code Generation	Produces target code

Detailed study of these phases comes in later lectures.

9. Optimization — Important Principle

An optimization should improve the program without changing its meaning.

For example, a compiler may replace an unnecessary computation with a simpler one.

However, a mathematically valid transformation is not automatically valid for every programming language or data type.

For example, **floating-point arithmetic** has special values such as NaN, so compiler optimizations must respect the actual semantics of the language.

GATE Principle

Optimization = Efficiency + Preservation of Program Semantics

11. Compiler Design: Theory + Engineering

Compiler design involves both:

Theory

- Programming languages
- Formal languages
- Grammars
- Algorithms
- Program analysis

Engineering

- Compiler implementation
- Efficient code generation
- Runtime systems
- Hardware considerations
- Performance

Therefore:

Compiler design is a combination of theoretical foundations and practical engineering.

12. Evolution of Compiler Design

This notes gives a high-level comparison between early and modern compilers.

Earlier compilers

Significant effort was required for:

- Lexical analysis
- Parsing
- Optimization
- Code generation

Modern compilers

Lexical analysis and parsing have become comparatively easier due to mature techniques and tools.

Modern compilers place much greater emphasis on:

- Semantic analysis
- Sophisticated optimization

Main Idea

The challenges in compiler design change as **programming languages and computer architectures evolve**.

13. Key Historical Idea

The development of compilers was important because it allowed programmers to work at a higher level of abstraction without completely sacrificing machine efficiency.

Machine-level programming



Difficult for programmers



High-level languages



Compiler



Efficient target program

This is one of the fundamental motivations behind compiler technology.

Code Generation → Target Program

COMPILER DESIGN

Core Concepts and Programming Language Design

The Five Phases of a Compiler

A compiler transforms a **source program** into a target program through a sequence of major phases.

Overall Flow

Source Program

↓

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis

↓

Optimization

↓

Code Generation

↓

Target Program

1. Lexical Analysis

Lexical analysis converts the sequence of characters in the source program into meaningful **tokens**.

Example

if (x == 10)

Possible tokens:

if (x == 10)

The lexical analyzer identifies elements such as:

- Keywords
- Identifiers

- Constants
- Operators
- Separators / punctuation

Important Point

A lexical analyzer must decide how characters are grouped into tokens.

For example:

==

may represent **one operator token**, rather than two separate = tokens.

GATE Focus:

Lexical Analysis → Characters → Tokens

2. Syntax Analysis / Parsing

Syntax analysis determines whether the sequence of tokens follows the **grammar of the programming language**.

It constructs a structural representation, commonly a **parse tree**.

Example

if condition then statement else statement

The parser determines how the different components are structurally related.

Main Purpose

- Checks syntactic structure.
- Uses the grammar of the language.
- Builds a parse tree or related representation.

GATE Focus:

Syntax Analysis → Tokens → Syntactic Structure

3. Semantic Analysis

Semantic analysis checks whether the program is **meaningful according to the rules of the language**.

Typical concerns include:

- Variable binding
- Scope
- Type checking
- Consistency of declarations and uses

Example — Scope

```
x = 10
{
  x = 20
}
```

The inner declaration/use of x can hide the outer x depending on the language's scope rules.

Thus, semantic analysis is concerned not merely with whether a program is grammatically correct, but whether its meaning is valid.

GATE Focus:

Semantic Analysis → Meaning / Consistency

4. Optimization

Optimization transforms the program into an **equivalent but more efficient form**.

The transformation must preserve the meaning of the program.

Possible Goals

- Reduce execution time
- Reduce memory usage
- Reduce power consumption
- Reduce communication/network cost
- Improve overall resource utilization

Example

For integer arithmetic:

$$x = y \times 0$$

can be simplified to:

$$x = 0$$

However, such transformations must consider the semantics of the data type.

For example, floating-point arithmetic may involve special values such as **NaN**, so the same transformation is not always valid.

Key Principle

Optimization changes the implementation, not the meaning.

GATE Focus:

Optimization → Equivalent program with better efficiency

5. Code Generation

Code generation converts the compiler's internal representation into a **target representation**.

The target may be:

- Machine code
- Assembly language
- Bytecode
- Another intermediate/target language

Basic Idea

Internal Representation



Code Generator



Target Program

GATE Focus:

Code Generation → Target Program

Compiler Phase Summary

Phase	Main Function
Lexical Analysis	Characters → Tokens
Syntax Analysis	Tokens → Syntactic Structure

Phase	Main Function
Semantic Analysis	Checks Meaning and Consistency
Optimization	Improves Efficiency
Code Generation	Produces Target Code

Easy Recall

L → S → S → O → C

Lexical → Syntax → Semantic → Optimization → Code Generation

Early vs Modern Compilers

The relative complexity of compiler phases has changed over time.

Early Compilers

Early compilers had significant complexity in:

- Lexical analysis
- Parsing
- Optimization
- Code generation

Semantic analysis was comparatively small.

Modern Compilers

Modern compiler design generally has:

- Less complexity in lexical analysis
- Less complexity in parsing due to mature tools and techniques
- Much greater semantic analysis
- Very large and sophisticated optimization
- Mature code generation

Key Idea

Compiler technology has evolved, so the **distribution of complexity among phases** has also changed.

The Economy of Programming Languages

Programming languages are designed and developed according to the needs of programmers and application domains.

There is **no single programming language that is ideal for every type of application**.

1. Different Domains, Different Requirements

Different applications require different programming features.

Scientific Computing

Typical requirements:

- Floating-point computation
- Arrays
- Numerical processing
- Parallel computation

A language such as **FORTRAN** was strongly influenced by these requirements.

Business Applications

Typical requirements:

- Data storage
- Data processing
- Report generation
- Large-scale data management

Database-oriented technologies and languages such as **SQL** are designed around such requirements.

Systems Programming

Typical requirements:

- Low-level control

Memory/resource managementIEEE Research-Paper Format: Student Notes

1. Typical Paper Structure

- **Title and author information**
- **Abstract**
- **Index Terms**
- **Introduction**
- **Related Work**
- **Methodology**
- **Results**
- **Discussion**
- **Conclusion**
- **References**

Follow the target IEEE journal or conference template because requirements may vary. Use the official template whenever one is provided.

2. Title and Author Information

- Write a specific, informative title that identifies the topic, method, system, or main contribution.
- Avoid unnecessary abbreviations, vague wording, and excessive length.
- List each author's full name and affiliation according to the template.
- Include institutional details, location, and contact information only when required.
- Check the author order and spelling carefully before submission.

3. Abstract

- Summarize the problem, approach, main results, and significance.
- Make the abstract understandable without requiring the full paper.
- Avoid citations, unexplained abbreviations, footnotes, and detailed equations.
- Follow the word limit specified by the publication.

4. Index Terms

Provide a short list of keywords that describe the paper's subject areas, methods, and applications. Use terms that readers are likely to search for, and follow the publication's punctuation and capitalization rules.

5. Section Headings

- Use a consistent hierarchy of numbered headings.
- Common primary headings include **I. INTRODUCTION**, **II. RELATED WORK**, and **III. METHODOLOGY**.
- Use secondary headings to divide major sections into logical subsections.
- Do not create a subsection unless the section contains at least two meaningful divisions.
- Keep headings concise and make them accurately describe the following content.

6. Introduction

- Introduce the broader problem and explain why it matters.
- Identify the research gap, limitation, or unresolved question.
- State the paper's objective and contributions clearly.
- Briefly describe the organization of the paper when appropriate.

7. Related Work

- Summarize important previous studies, systems, or theories.
- Compare earlier approaches rather than merely listing them.
- Explain how the present work differs from or extends prior research.
- Use accurate citations for every borrowed idea, result, method, or quotation.

8. Methodology

- Describe the research design, materials, data, tools, and procedures.
- Provide enough detail for another researcher to understand or reproduce the work.
- Define variables, assumptions, datasets, evaluation metrics, and experimental settings.
- Explain ethical, privacy, or safety considerations when relevant.

9. Results

- Present findings objectively and in a logical order.
- Use figures and tables to make patterns, comparisons, and numerical results clear.
- Report appropriate measurements, uncertainty, sample sizes, or statistical tests.
- Avoid repeating every value from a table in the surrounding paragraph.

10. Discussion

- Interpret the results and connect them to the research question.
- Explain whether the findings support or challenge earlier work.
- Address unexpected outcomes and possible reasons for them.

- Discuss limitations, practical implications, and possible sources of bias.

11. Conclusion

- Restate the central contribution without copying the abstract.
- Summarize the most important findings and their significance.
- Do not introduce major new evidence or uncited claims.
- Suggest focused directions for future research when appropriate.

12. Equations

- Introduce each important equation in the surrounding text.
- Number equations consecutively when they are referenced later.
- Define every symbol and explain variables immediately after the equation or before its use.
- Use consistent notation, italicizing mathematical variables according to the template.
- Refer to equations as “(1)” or according to the required IEEE convention.

13. Figures and Tables

- Number figures and tables in the order in which they are first mentioned.
- Refer to every figure and table in the text before it appears.
- Use descriptive captions; IEEE conventions commonly place figure captions below figures and table titles above tables.
- Ensure labels, legends, units, and text remain readable at publication size.
- Do not duplicate the same information unnecessarily in prose, figures, and tables.
- Use high-quality graphics and obtain permission for material that is not yours.

14. In-Text Citations

- IEEE commonly uses numbered citations in square brackets, such as [1] or [2], [3].
- Assign numbers according to the order in which sources first appear.
- Use the same number whenever you cite the same source again.
- Place citations near the relevant claim, statement, or quotation.
- Cite original sources whenever possible and verify that every citation has a matching reference entry.

15. IEEE Reference List Style

- Arrange references numerically according to citation order, not alphabetically.
- Include the authors, title, publication or venue, volume or issue when applicable, page range, date, and DOI or URL when required.
- Use the prescribed IEEE abbreviations and punctuation consistently.
- Format different source types—journal articles, conference papers, books, websites, standards, and theses—according to their respective IEEE examples.

- Check author names, capitalization, publication details, and links carefully.

16. Formatting Conventions

- Use the official IEEE template and do not alter its margins, column layout, fonts, spacing, or heading styles without permission.
- Maintain consistent capitalization, terminology, abbreviations, symbols, and units.
- Define an abbreviation at first use, followed by the abbreviation in parentheses.
- Use the required units, commonly SI units, and state units clearly.
- Check page limits, file type, anonymization rules, copyright forms, and supplementary-material requirements.
- Proofread for grammar, spelling, transitions, and consistency between text, figures, tables, equations, and references.

17. Final Checklist

- Is the title specific and consistent with the paper's content?
- Are all authors, affiliations, and required contact details correct?
- Does the abstract state the problem, approach, results, and contribution?
- Are the index terms relevant and formatted correctly?
- Does every major section have a clear purpose and logical order?
- Are the methodology and evaluation detailed enough to support reproducibility?
- Are results separated from interpretation, with limitations discussed?
- Are all equations, figures, and tables numbered, introduced, and explained?
- Does every in-text citation match one reference-list entry?
- Are references numbered and formatted in IEEE style?
- Does the manuscript follow the official template and submission requirements?
- Has the paper been proofread and checked for plagiarism, unsupported claims, and accidental omissions?
- These notes provide general guidance for preparing an IEEE-style research paper. Always compare the completed manuscript with the specific instructions supplied by the target IEEE publication.
- Hardware interaction
- Operating-system development
- Embedded systems
- Timing and performance control

Languages such as **C/C++** are widely associated with these requirements.

2. Why Do New Programming Languages Appear?

New programming languages often arise because:

1. Existing languages do not adequately support a new application domain.
2. New technologies create new programming requirements.
3. Developers seek higher productivity.
4. A new language can provide better abstractions for a particular problem.

Central Idea

New technology or a new application domain → New requirements → Opportunity for a new language

3. The Cost of Learning a Language

One of the most important economic factors in programming languages is the **cost of training programmers**.

Learning a new language requires:

- Time
- Training
- Practice
- Documentation
- Development experience

Therefore, changing an established language can be expensive when millions of programmers already use it.

4. Why Popular Languages Change Slowly

Widely used programming languages have a large existing user base.

Major changes can require programmers to:

- Learn new concepts
- Update existing knowledge
- Modify development practices
- Rewrite or adapt existing programs

Therefore, established languages tend to evolve **more slowly**.

Important Concept

Large user base → High training/adoption cost → Slower evolution

This can cause a language to become relatively **ossified** over time.

5. Why New Languages Can Evolve Quickly

A new language initially has relatively few users.

Therefore:

Small user base → Lower adoption cost → Faster experimentation and evolution

New languages can therefore introduce major design changes more easily.

6. Programmer Productivity vs Training Cost

A programmer may choose a new language when the expected productivity improvement is greater than the cost of learning it.

Simplified Relationship

Adoption becomes attractive when:

Productivity Gain > Training / Switching Cost

This is one reason programmers do not automatically move to every technically interesting new language.

7. Similarity Between New and Existing Languages

New languages often resemble already familiar languages.

Example

A language designed with concepts or syntax familiar to programmers can reduce the learning curve.

For example, **Java** shares many familiar characteristics with **C/C++**.

Why?

Because similarity reduces:

- Training cost

- Learning time
 - Adoption barriers
-

8. What Makes a Programming Language Successful?

There is no single universal measure of a "good" programming language.

A language's success can depend on:

- Application domain
- Programmer productivity
- Ease of learning
- Existing ecosystem
- Adoption cost
- Historical factors
- Availability of tools and libraries

Important Point

Popularity does not necessarily mean technical superiority.

A language may remain popular partly because of:

- Existing programs
 - Existing developers
 - Existing tools
 - Industry adoption
 - Historical momentum
-

9. Key Economic Principle

Programming language design is not purely a technical problem.

It involves a combination of:

Technology + Programmer Productivity + Training Cost + Adoption + Historical Factors

Compiler

Source Program → Compiler → Target Program

Five Major Phases

L → S → S → O → C

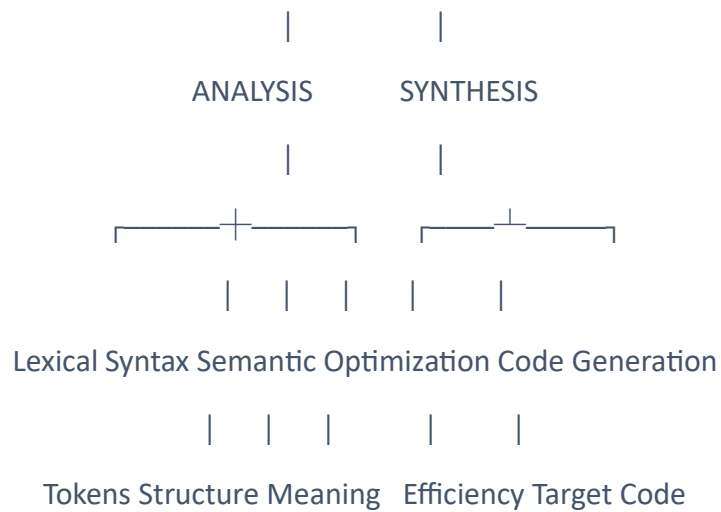
Phase	Remember
Lexical	Characters → Tokens
Syntax	Tokens → Structure
Semantic	Meaning / Consistency
Optimization	Improve efficiency while preserving meaning
Code Generation	Produce target code

Programming Languages

- Different domains have different requirements.
- No single language is best for every domain.
- Scientific computing → numerical/array-oriented requirements.
- Business applications → data processing/reporting.
- Systems programming → low-level control and resource management.
- Programmer training is a major economic cost.
- Large user bases make major language changes expensive.
- Established languages tend to evolve slowly.
- New languages can evolve rapidly because they have fewer users.
- New languages often resemble existing languages to reduce learning cost.
- Popularity alone does not determine technical quality.

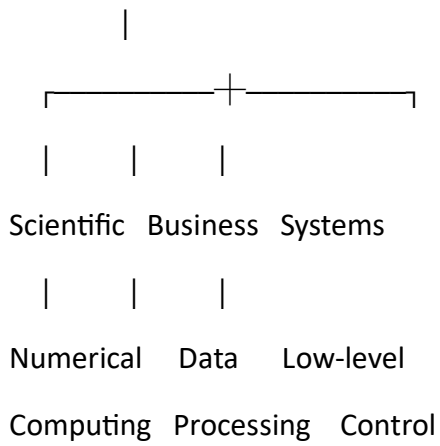
ONE-PAGE MEMORY MAP





PROGRAMMING LANGUAGES

Different Domains



New Requirements

New Languages

Productivity vs Training Cost

Adoption & Evolution

Final GATE Takeaway

Compiler design is both a technical and engineering discipline, while programming-language evolution is influenced not only by technical requirements but also by productivity, training cost, adoption, and historical factors.